DÍA+

Estructura del proyecto a futuro y función de cada parte
Aplicación móvil Android · Kotlin · Jetpack Compose

1. Objetivo del documento

Este documento describe cómo está organizado hoy el proyecto **DÍA+** (una app de tareas y recordatorios) y cómo se organizaría cuando pase de ser un **diseño de interfaz** a una **aplicación funcional**. Para cada carpeta y archivo se explica qué responsabilidad tiene y por qué existe.

> **Estado actual:** el proyecto contiene la capa visual (pantallas, componentes y colores) con datos de ejemplo. Las tareas no se guardan y no hay notificaciones reales. La estructura de este documento indica dónde iría cada funcionalidad nueva.

2. Tecnologías elegidas

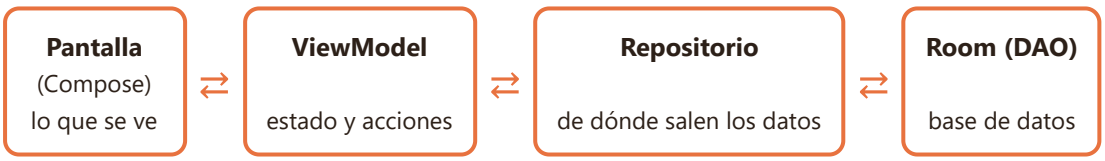
Tecnología	Para qué se usa	Por qué
Kotlin	Lenguaje de programación	Es el lenguaje recomendado por Google para Android. Es más corto y seguro que Java (evita muchos errores de `null`) y es compatible con todo el código Java existente.
Jetpack Compose	Construir la interfaz	Es el sistema moderno de interfaces de Android. Se escribe la pantalla en Kotlin (sin archivos XML de diseño) y permite ver el resultado con `@Preview` sin ejecutar la app.
Material 3	Componentes base (campos de texto, botones, switch)	Viene incluido con Compose, sigue las guías de diseño de Android y se puede personalizar con los colores propios del diseño.
Gradle (Kotlin DSL)	Compilar y gestionar librerías	Es el sistema de compilación oficial de Android. Las versiones se centralizan en `libs.versions.toml`.
Room FUTURO	Guardar las tareas en el teléfono	Base de datos SQLite oficial de Android, con comprobación de las consultas al compilar.
ViewModel FUTURO	Manejar el estado de cada pantalla	Conserva los datos al girar el teléfono y separa la lógica de la interfaz.
WorkManager / AlarmManager FUTURO	Recordatorios a la hora de la tarea	Permiten lanzar notificaciones aunque la app esté cerrada.
Navigation Compose FUTURO	Moverse entre pantallas	Forma oficial de definir rutas entre pantallas en Compose.

3. Estructura actual

```
DiaPlus/
├── app/
│   ├── build.gradle.kts                configuración del módulo de la app
│   └── src/main/
│       ├── AndroidManifest.xml         "carnet" de la app
│       ├── java/com/diaplus/app/
│       │   ├── MainActivity.kt         punto de entrada
│       │   ├── data/
│       │   │   └── Tarea.kt            modelo de tarea + tareas de ejemplo
│       │   └── ui/
│       │       ├── Previews.kt         vistas previas de pantallas completas
│       │       ├── theme/Colores.kt    colores claro y oscuro
│       │       ├── screens/            PantallaPrincipal.kt, Bienvenida.kt
│       │       └── components/         TarjetaTarea, Encabezado, CampoAnotar,
│       │                               FormularioTarea, Botones
│       └── res/                        iconos (drawable), textos, tema base
├── gradle/libs.versions.toml           versiones de librerías
├── build.gradle.kts / settings.gradle.kts
└── local.properties                    ruta del SDK en este PC
```

4. Arquitectura a futuro: MVVM

Cuando la app sea funcional se propone seguir el patrón **MVVM (Model – View – ViewModel)**, que es el que recomienda Google para Android. La idea es separar el proyecto en **capas**, donde cada una tiene una sola responsabilidad y solo habla con la capa vecina:



- **La pantalla** solo dibuja y avisa de lo que hace el usuario ("tocó el +", "marcó una tarea").
- **El ViewModel** recibe esos avisos, decide qué hacer y le entrega a la pantalla un **estado** listo para dibujar.
- **El repositorio** es el único que sabe si los datos vienen de la base de datos, de internet o de ambos.
- **Room** guarda y lee las tareas en el teléfono.

Ventaja: si mañana se cambia la base de datos, o se agrega sincronización con internet, las pantallas no se tocan. Además, cada capa se puede probar por separado.

5. Estructura propuesta a futuro

```
app/src/main/java/com/diaplus/app/
├── DiaPlusApp.kt                    Application: se crea una vez al abrir la app
├── MainActivity.kt                  arranca Compose y la navegación
│
├── navigation/
│   ├── Rutas.kt                     nombres de las pantallas ("inicio", "tarea/{id}"…)
│   └── NavegacionApp.kt             NavHost: qué pantalla corresponde a cada ruta
│
├── data/                            ── CAPA DE DATOS ──
│   ├── model/
│   │   └── Tarea.kt                 la tarea como entidad de Room (@Entity)
│   ├── local/
│   │   ├── TareaDao.kt              consultas: insertar, actualizar, borrar, listar
│   │   ├── BaseDeDatos.kt           base de datos Room de la app
│   │   └── Preferencias.kt          ajustes simples (modo oscuro) con DataStore
│   └── repository/
│       └── TareaRepositorio.kt      punto único de acceso a las tareas
│
├── domain/                          ── LÓGICA DE NEGOCIO (opcional) ──
│   ├── LeerTextoNatural.kt          "entregar mañana 3pm" → título, fecha y hora
│   ├── CalcularRacha.kt             días seguidos completando tareas
│   └── TextoDeTiempo.kt             "en 2h", "hace 3d", "vencida"
│
├── notifications/                   ── RECORDATORIOS ──
│   ├── ProgramadorAvisos.kt         programa el aviso a la hora (y 30 min antes)
│   ├── ReceptorAviso.kt             BroadcastReceiver que se despierta a la hora
│   └── CanalNotificaciones.kt       crea el canal y construye la notificación
│
└── ui/                              ── CAPA DE INTERFAZ ──
    ├── theme/
    │   ├── Colores.kt               colores claro / oscuro (ya existe)
    │   ├── Tipografia.kt            fuentes Nunito y Outfit
    │   └── Tema.kt                  junta colores y fuentes
    ├── components/                  piezas reutilizables (ya existen)
    │   ├── TarjetaTarea.kt  Encabezado.kt  CampoAnotar.kt
    │   ├── FormularioTarea.kt  Botones.kt
    │   └── AvisoDeshacer.kt         aviso "Deshacer" tras completar o borrar
    └── screens/
        ├── bienvenida/
        │   └── Bienvenida.kt
        ├── inicio/
        │   ├── PantallaPrincipal.kt     solo dibuja
        │   ├── InicioViewModel.kt       estado y acciones de la pantalla
        │   └── InicioEstado.kt          datos que necesita la pantalla
        ├── tarea/
        │   ├── PantallaTarea.kt         crear / editar tarea
        │   └── TareaViewModel.kt
        └── ajustes/
            ├── PantallaAjustes.kt       tema, sonido de avisos, etc.
            └── AjustesViewModel.kt

app/src/main/res/
├── drawable/      iconos vectoriales (ya existen)
├── font/          nunito_*.ttf, outfit_*.ttf
├── values/        strings.xml (todos los textos), colors.xml, themes.xml
```

```
└── xml/           reglas de copia de seguridad

app/src/test/          pruebas unitarias (lógica: leer texto, racha, ViewModels)
app/src/androidTest/   pruebas de interfaz y de la base de datos

wear/                  (módulo futuro) versión para reloj Wear OS
```

6. Función de cada parte

Raíz de la app

Parte	Función
`DiaPlusApp.kt` FUTURO	Clase `Application`. Se ejecuta una sola vez al abrir la app; ahí se crean la base de datos, el repositorio y el canal de notificaciones para compartirlos con todas las pantallas.
`MainActivity.kt` EXISTE	Puerta de entrada. Hoy muestra la bienvenida y luego la pantalla principal; a futuro solo cargará el tema y la navegación.
`AndroidManifest.xml` EXISTE	Declara el nombre, el icono, la primera pantalla y, a futuro, los permisos (`POST_NOTIFICATIONS`, alarmas exactas) y el receptor de avisos.

navigation/ FUTURO

Parte	Función
`Rutas.kt`	Lista de nombres de pantallas en un solo lugar, para no escribir textos sueltos y evitar errores al navegar.
`NavegacionApp.kt`	Contiene el `NavHost`: dice qué pantalla se muestra en cada ruta y cómo se pasa de una a otra (por ejemplo, de la lista a editar una tarea con su id).

data/ — capa de datos

Parte	Función
`model/Tarea.kt` EXISTE	Define qué es una tarea. Hoy guarda textos fijos; a futuro tendrá `id`, fecha y hora reales y se marcará con `@Entity` para que Room cree la tabla.
`local/TareaDao.kt`	"Data Access Object": las consultas a la tabla de tareas (insertar, actualizar, borrar, listar pendientes ordenadas por fecha).
`local/BaseDeDatos.kt`	Crea la base de datos Room y entrega el DAO.
`local/Preferencias.kt`	Guarda ajustes pequeños como el modo oscuro, usando DataStore.
`repository/TareaRepositorio.kt`	Único punto por el que la app pide o guarda tareas. Esconde si los datos vienen de Room o, más adelante, de internet.

domain/ — lógica de negocio FUTURO

Parte	Función

`LeerTextoNatural.kt`	Interpreta lo escrito en "Anotar algo…": detecta "hoy", "mañana", días de la semana y horas como "3pm" o "a las 8:30".
`CalcularRacha.kt`	Cuenta cuántos días seguidos se completó al menos una tarea (la racha del diseño).
`TextoDeTiempo.kt`	Convierte fechas en textos cortos: "en 2h", "mañana", "hace 3d", y decide si una tarea está vencida.

Esta capa no depende de Android, así que es la más fácil de probar con pruebas unitarias.

notifications/ — recordatorios FUTURO

Parte	Función
`ProgramadorAvisos.kt`	Al crear o editar una tarea con hora, programa el aviso a esa hora y, si está activado, otro 30 minutos antes. Al completarla o borrarla, los cancela.
`ReceptorAviso.kt`	Se ejecuta cuando llega la hora, incluso con la app cerrada, y muestra la notificación.
`CanalNotificaciones.kt`	Crea el canal de notificaciones (obligatorio desde Android 8) y arma la notificación con el título de la tarea y el botón "Ya quedó".

ui/ — capa de interfaz

Parte	Función
`theme/` EXISTE	Colores (y a futuro fuentes y tema) de toda la app. Cambiar un color aquí lo cambia en todas las pantallas.
`components/` EXISTE	Piezas reutilizables que no saben de dónde salen los datos: reciben lo que deben mostrar y avisan cuando el usuario las toca. Cada una tiene su `@Preview`.
`screens/*/Pantalla….kt`	Arman una pantalla completa con los componentes. Solo dibujan el estado que les da su ViewModel.
`screens/*/…ViewModel.kt` FUTURO	Guardan el estado de la pantalla, reciben las acciones del usuario (marcar, borrar, guardar) y llaman al repositorio.
`screens/*/…Estado.kt` FUTURO	Clase con todo lo que la pantalla necesita mostrar: lista de pendientes, completadas, saludo, racha, si el formulario está abierto.
`Previews.kt` EXISTE	Vistas previas de las pantallas completas para revisar el diseño sin ejecutar la app.

res/, pruebas y otros

Parte	Función
`res/drawable/`	Iconos vectoriales en XML (campana, luna, sol, calendario, reloj, +, logo).
`res/font/` FUTURO	Archivos de las fuentes Nunito y Outfit del diseño.
`res/values/strings.xml`	Todos los textos de la app. Tenerlos aquí permite traducir la app a otros idiomas sin tocar el código.

`test/` y `androidTest/` FUTURO	Pruebas automáticas: `test` para la lógica (se ejecuta en el PC) y `androidTest` para la interfaz y la base de datos (se ejecuta en un emulador).
`wear/` FUTURO	Módulo aparte para el reloj (Wear OS) con las pantallas "Aviso" y "Lista de hoy" del diseño; compartiría los datos con el teléfono.
Archivos de Gradle	`build.gradle.kts`, `settings.gradle.kts`, `libs.versions.toml`: qué librerías usa el proyecto, en qué versión y cómo se compila.

7. Cómo fluye una acción (ejemplo)

El usuario escribe "entregar ensayo mañana 3pm" y pulsa guardar:

1. `CampoAnotar` (componente) avisa: "el usuario envió este texto".
2. `InicioViewModel` recibe el texto y usa `LeerTextoNatural` → título "entregar ensayo", fecha mañana, hora 15:00.
3. El ViewModel pide a `TareaRepositorio` que guarde la tarea.
4. El repositorio la inserta con `TareaDao` en Room y llama a `ProgramadorAvisos` para el recordatorio.
5. Room avisa que la lista cambió → el ViewModel actualiza el estado → `PantallaPrincipal` se vuelve a dibujar con la nueva tarjeta.
6. Mañana a las 15:00, `ReceptorAviso` muestra la notificación aunque la app esté cerrada.

8. Plan de trabajo por fases

Fase	Qué se hace	Partes de la estructura
1 HECHO	Diseño de interfaz con datos de ejemplo y vistas previas	`ui/theme`, `ui/components`, `ui/screens`, `res/drawable`
2	Fuentes del diseño y textos en `strings.xml`	`res/font`, `ui/theme/Tipografia.kt`, `res/values`
3	Guardar tareas de verdad	`data/model`, `data/local`, `data/repository`
4	Separar lógica de la pantalla	`InicioViewModel`, `InicioEstado`
5	Fecha y hora reales, "mañana 3pm", vencidas, racha	`domain/` + pruebas en `test/`
6	Crear / editar / borrar con gestos y "Deshacer"	`screens/tarea`, `navigation/`, `AvisoDeshacer`
7	Recordatorios con notificaciones	`notifications/`, permisos en el Manifest
8	Ajustes y modo oscuro guardado	`screens/ajustes`, `Preferencias.kt`
9	Versión para reloj	módulo `wear/`

9. Justificación de las decisiones de estructura

Decisión	Justificación
Separar `ui`, `data` y `domain`	Cada carpeta tiene una sola responsabilidad. Un error de diseño se busca en `ui`, un error al guardar en `data`, un error de cálculo en `domain`.
Componentes pequeños con `@Preview`	Se reutilizan en varias pantallas (la tarjeta aparece en pendientes y completadas) y cada uno se revisa visualmente por separado sin ejecutar la app.

Colores en un solo archivo	El modo claro y oscuro se cambian en un único lugar, y se asegura que toda la app use los colores exactos del diseño de Figma.
Iconos como vectores XML	Se ven nítidos en cualquier tamaño de pantalla, pesan muy poco y se les cambia el color con `tint` según el tema.
MVVM con ViewModel	Patrón recomendado por Google; los datos sobreviven al girar el teléfono y la pantalla no mezcla lógica con diseño.
Repositorio entre ViewModel y Room	Si se agrega sincronización con internet, solo cambia el repositorio; el resto de la app sigue igual.
Lógica en `domain` sin Android	Se puede probar con pruebas unitarias rápidas en el PC, sin emulador.
Módulo aparte para el reloj	El reloj tiene otra pantalla (circular) y otras reglas de diseño; separarlo evita mezclar código de dos dispositivos.